

ISM4641: Python for Business Analytics

Lecture 3: Data Structures - Tuples and Sets

Dr. Smith

2025-08-25

Welcome to Week 3!

Today's Learning Journey

New Data Structures

- ▶ **Tuples:** Immutable, ordered collections
- ▶ **Sets:** Unique elements, unordered
- ▶ **When to use each type**

Functions

- ▶ **Creating reusable code**
- ▶ **Parameters and return values**
- ▶ **Variable scope concepts**
- ▶ **Lambda functions**

Note

This week introduces advanced data structures and the foundation of modular programming!

Part 1: Tuples

Introduction to Tuples

Tuples are immutable, ordered collections

► **Ordered:** Elements maintain their position

```
1 # Creating tuples
2 empty_tuple = ()
3 product_info = ("Laptop", "Dell XPS 13", 999.99, True)
4 coordinates = 40.7128, -74.0060 # No parentheses needed
5 single_item = ("Apple",) # Note the comma for single items
6
7 print(f"Product: {product_info}")
8 print(f"NYC Coordinates: {coordinates}")
```

Product: ('Laptop', 'Dell XPS 13', 999.99, True)

NYC Coordinates: (40.7128, -74.006)

Introduction to Tuples

Tuples are immutable, ordered collections

- ▶ **Ordered:** Elements maintain their position
- ▶ **Immutable:** Cannot be changed after creation

```
1 # Creating tuples
2 empty_tuple = ()
3 product_info = ("Laptop", "Dell XPS 13", 999.99, True)
4 coordinates = 40.7128, -74.0060 # No parentheses needed
5 single_item = ("Apple",) # Note the comma for single items
6
7 print(f"Product: {product_info}")
8 print(f"NYC Coordinates: {coordinates}")
```

Product: ('Laptop', 'Dell XPS 13', 999.99, True)

NYC Coordinates: (40.7128, -74.006)

Introduction to Tuples

Tuples are immutable, ordered collections

- ▶ **Ordered:** Elements maintain their position
- ▶ **Immutable:** Cannot be changed after creation
- ▶ **Hashable:** Can be used as dictionary keys

```
1 # Creating tuples
2 empty_tuple = ()
3 product_info = ("Laptop", "Dell XPS 13", 999.99, True)
4 coordinates = 40.7128, -74.0060 # No parentheses needed
5 single_item = ("Apple",) # Note the comma for single items
6
7 print(f"Product: {product_info}")
8 print(f"NYC Coordinates: {coordinates}")
```

Product: ('Laptop', 'Dell XPS 13', 999.99, True)

NYC Coordinates: (40.7128, -74.006)

Introduction to Tuples

Tuples are immutable, ordered collections

- ▶ **Ordered:** Elements maintain their position
- ▶ **Immutable:** Cannot be changed after creation
- ▶ **Hashable:** Can be used as dictionary keys
- ▶ **Parentheses:** Created using `()` or comma separation

```
1 # Creating tuples
2 empty_tuple = ()
3 product_info = ("Laptop", "Dell XPS 13", 999.99, True)
4 coordinates = 40.7128, -74.0060 # No parentheses needed
5 single_item = ("Apple",) # Note the comma for single items
6
7 print(f"Product: {product_info}")
8 print(f"NYC Coordinates: {coordinates}")
```

Product: ('Laptop', 'Dell XPS 13', 999.99, True)

NYC Coordinates: (40.7128, -74.006)

Tuple Operations

Indexing and Slicing

```
1 # Business container data
2 container_data = ("CONT001", "Miami", "New York", 15000.50)
3
4 # Accessing elements
5 container_id = container_data[0]
6 origin = container_data[1]
7 destination = container_data[2]
8 value = container_data[3]
9
10 print(f"Container {container_id}: {origin} → {destination}")
11 print(f"Cargo Value: ${value:,.2f}")
12
13 # Slicing operations
14 route_info = container_data[1:3] # Origin and destination
15 print(f"Route: {route_info}")
```

Tuple Unpacking

Elegant Variable Assignment

```
1 # Employee record
2 employee = ("Alice Johnson", "Marketing", 65000, 3)
3
4 # Unpacking into variables
5 name, department, salary, years = employee
6
7 print(f"Employee: {name}")
8 print(f"Department: {department}")
9 print(f"Salary: ${salary:,}")
10 print(f"Years of Service: {years}")
11
12 # Partial unpacking with underscore for unused values
13 employee_name, _, employee_salary, _ = employee
14 print(f"{employee_name} earns ${employee_salary:,}")
```

Employee: Alice Johnson

Tuple Methods

```
1  # Customer transaction history
2  transactions = (250.00, 89.99, 250.00, 134.50, 250.00, 76.2
3
4  # Count occurrences
5  frequent_amount = transactions.count(250.00)
6  print(f"Transactions of $250.00: {frequent_amount}")
7
8  # Find index of first occurrence
9  first_250_index = transactions.index(250.00)
10 print(f"First $250 transaction at position: {first_250_index}")
11
12 # Total transactions and sum
13 total_transactions = len(transactions)
14 total_amount = sum(transactions)
15 avg_amount = total_amount / total_transactions
16
17 print(f"Total Transactions: {total_transactions}")
18 print(f"Average Transaction: ${avg_amount:.2f}")
```

Business Applications of Tuples

Fixed Data Records

```
1 # Product catalog - data that shouldn't change
2 products = [
3     ("SKU001", "Wireless Headphones", 149.99, "Electronics", False),
4     ("SKU002", "Coffee Mug", 12.99, "Home", True),
5     ("SKU003", "Smartphone Case", 24.99, "Electronics", False),
6     ("SKU004", "Desk Lamp", 89.99, "Office", True)
7 ]
8
9 print("Product Catalog:")
10 print("-" * 60)
11 for sku, name, price, category, in_stock in products:
12     status = " " if in_stock else " "
13     print(f"{sku}: {name:<20} ${price:>6.2f} [{category}] {status}")
```

Product Catalog:

Part 2: Sets

Introduction to Sets

Sets store unique, unordered collections

► **Unique Elements:** Automatically removes duplicates

```
1 # Creating sets
2 empty_set = set() # Note: {} creates a dict, not a set
3 customer_ids = {101, 102, 103, 104, 105}
4 mixed_set = {1, "hello", 3.14, True}
5
6 # From lists (removes duplicates)
7 sales_reps = ["Alice", "Bob", "Charlie", "Alice", "Diana",
8 unique_reps = set(sales_reps)
9
10 print(f"Sales reps with duplicates: {sales_reps}")
11 print(f"Unique sales reps: {unique_reps}")
```


Introduction to Sets

Sets store unique, unordered collections

- ▶ **Unique Elements:** Automatically removes duplicates
- ▶ **Unordered:** No index-based access

```
1 # Creating sets
2 empty_set = set() # Note: {} creates a dict, not a set
3 customer_ids = {101, 102, 103, 104, 105}
4 mixed_set = {1, "hello", 3.14, True}
5
6 # From lists (removes duplicates)
7 sales_reps = ["Alice", "Bob", "Charlie", "Alice", "Diana",
8 unique_reps = set(sales_reps)
9
10 print(f"Sales reps with duplicates: {sales_reps}")
11 print(f"Unique sales reps: {unique_reps}")
```

Introduction to Sets

Sets store unique, unordered collections

- ▶ **Unique Elements:** Automatically removes duplicates
- ▶ **Unordered:** No index-based access
- ▶ **Mutable:** Can add/remove elements

```
1 # Creating sets
2 empty_set = set() # Note: {} creates a dict, not a set
3 customer_ids = {101, 102, 103, 104, 105}
4 mixed_set = {1, "hello", 3.14, True}
5
6 # From lists (removes duplicates)
7 sales_reps = ["Alice", "Bob", "Charlie", "Alice", "Diana",
8 unique_reps = set(sales_reps)
9
10 print(f"Sales reps with duplicates: {sales_reps}")
11 print(f"Unique sales reps: {unique_reps}")
```

Introduction to Sets

Sets store unique, unordered collections

- ▶ **Unique Elements:** Automatically removes duplicates
- ▶ **Unordered:** No index-based access
- ▶ **Mutable:** Can add/remove elements
- ▶ **Fast Lookups:** $O(1)$ average case for membership testing

```
1 # Creating sets
2 empty_set = set() # Note: {} creates a dict, not a set
3 customer_ids = {101, 102, 103, 104, 105}
4 mixed_set = {1, "hello", 3.14, True}
5
6 # From lists (removes duplicates)
7 sales_reps = ["Alice", "Bob", "Charlie", "Alice", "Diana",
8 unique_reps = set(sales_reps)
9
10 print(f"Sales reps with duplicates: {sales_reps}")
11 print(f"Unique sales reps: {unique_reps}")
```

Set Operations

Mathematical Set Operations

```
1 # Marketing campaign audiences
2 email_subscribers = {"Alice", "Bob", "Charlie", "Diana", "Eve"}
3 social_media_followers = {"Bob", "Charlie", "Grace", "Henry"}
4
5 print(f"Email subscribers: {email_subscribers}")
6 print(f"Social media followers: {social_media_followers}")
7 print()
8
9 # Union - all customers from either campaign
10 all_customers = email_subscribers | social_media_followers
11 print(f"All customers: {all_customers}")
12 print(f"Total reach: {len(all_customers)} customers")
13 print()
14
15 # Intersection - customers on both lists
16 engaged_customers = email_subscribers & social_media_followers
```

Set Operations (continued)

```
1 # Difference - customers only on email list
2 email_only = email_subscribers - social_media_followers
3 print(f"Email-only customers: {email_only}")
4
5 # Symmetric difference - customers on exactly one list
6 single_platform = email_subscribers ^ social_media_followers
7 print(f"Single platform customers: {single_platform}")
8
9 # Business insights
10 engagement_rate = len(engaged_customers) / len(all_customers)
11 print(f"\nCross-platform engagement rate: {engagement_rate}")
```

Email-only customers: {'Frank', 'Eve', 'Diana'}

Single platform customers: {'Ivan', 'Frank', 'Diana', 'Grace'}

Cross-platform engagement rate: 33.3%

Set Methods

Adding and Removing Elements

```
1  # Active customer tracking
2  active_customers = {"C001", "C002", "C003"}
3  print(f"Initially active: {active_customers}")
4
5  # Add single customer
6  active_customers.add("C004")
7  print(f"After adding C004: {active_customers}")
8
9  # Add multiple customers
10 new_customers = ["C005", "C006", "C007"]
11 active_customers.update(new_customers)
12 print(f"After bulk update: {active_customers}")
13
14 # Remove customers (different methods)
15 active_customers.remove("C002")  # Raises error if not present
16 active_customers.discard("C999")  # No error if not present
```

Business Set Applications

Customer Analysis

```
1 # Customer segmentation analysis
2 q1_customers = {"A101", "A102", "A103", "A104", "A105", "A106"}
3 q2_customers = {"A102", "A104", "A107", "A108", "A109", "A110"}
4 premium_customers = {"A101", "A104", "A107", "A111"}
5
6 print("Customer Analysis Report")
7 print("=" * 30)
8
9 # Loyal customers (purchased in both quarters)
10 loyal = q1_customers & q2_customers
11 print(f"Loyal customers (Q1 & Q2): {loyal}")
12 print(f"Loyalty rate: {len(loyal)} / {len(q1_customers | q2_customers)}")
13
14 # Customer acquisition and churn
15 new_acquisitions = q2_customers - q1_customers
16 churned_customers = q1_customers - q2_customers
```


Part 3: Functions

Why Functions Matter

Functions are the building blocks of professional programming:

- ▶ **Code Reusability:** Write once, use many times

Business Benefits

- ▶ Consistent calculations across reports
- ▶ Reduced errors from copy-paste code
- ▶ Faster development of new features
- ▶ Easier to update business rules

Why Functions Matter

Functions are the building blocks of professional programming:

- ▶ **Code Reusability:** Write once, use many times
- ▶ **Organization:** Break complex problems into manageable pieces

Business Benefits

- ▶ Consistent calculations across reports
- ▶ Reduced errors from copy-paste code
- ▶ Faster development of new features
- ▶ Easier to update business rules

Why Functions Matter

Functions are the building blocks of professional programming:

- ▶ **Code Reusability:** Write once, use many times
- ▶ **Organization:** Break complex problems into manageable pieces
- ▶ **Testing:** Easier to test small, focused functions

Business Benefits

- ▶ Consistent calculations across reports
- ▶ Reduced errors from copy-paste code
- ▶ Faster development of new features
- ▶ Easier to update business rules

Why Functions Matter

Functions are the building blocks of professional programming:

- ▶ **Code Reusability:** Write once, use many times
- ▶ **Organization:** Break complex problems into manageable pieces
- ▶ **Testing:** Easier to test small, focused functions
- ▶ **Maintenance:** Changes in one place affect all uses

Business Benefits

- ▶ Consistent calculations across reports
- ▶ Reduced errors from copy-paste code
- ▶ Faster development of new features
- ▶ Easier to update business rules

Why Functions Matter

Functions are the building blocks of professional programming:

- ▶ **Code Reusability:** Write once, use many times
- ▶ **Organization:** Break complex problems into manageable pieces
- ▶ **Testing:** Easier to test small, focused functions
- ▶ **Maintenance:** Changes in one place affect all uses
- ▶ **Collaboration:** Clear interfaces between team members

Business Benefits

- ▶ Consistent calculations across reports
- ▶ Reduced errors from copy-paste code
- ▶ Faster development of new features
- ▶ Easier to update business rules

Function Basics

Defining and Calling Functions

```
1 # Simple function without parameters
2 def greet_customer():
3     """Display a welcome message for customers."""
4     print("Welcome to TechCorps Analytics!")
5     print("We're here to help with your data needs.")
6
7 # Call the function
8 greet_customer()
```

Welcome to TechCorps Analytics!

We're here to help with your data needs.

```
1 # Function with parameters
2 def calculate_discount(price, discount_rate):
3     """Calculate discounted price for a product.
4
```

Function Parameters

Default Parameters

```
1 def calculate_loan_payment(principal, annual_rate=0.05, years=30):
2     """Calculate monthly loan payment using standard formula
3
4     Args:
5         principal (float): Loan amount
6         annual_rate (float): Annual interest rate (default 0.05)
7         years (int): Loan term in years (default 30)
8
9     Returns:
10        float: Monthly payment amount
11    """
12    monthly_rate = annual_rate / 12
13    num_payments = years * 12
14
15    if annual_rate == 0:
16        return principal / num_payments
```


Return Values

Single and Multiple Returns

```
1 def analyze_sales_performance(monthly_sales):
2     """Analyze monthly sales data and return key metrics.
3
4     Args:
5         monthly_sales (list): List of monthly sales figures
6
7     Returns:
8         tuple: (total, average, best_month, worst_month)
9     """
10    total_sales = sum(monthly_sales)
11    average_sales = total_sales / len(monthly_sales)
12    best_month = max(monthly_sales)
13    worst_month = min(monthly_sales)
14
15    return total_sales, average_sales, best_month, worst_month
```

Variable Scope

Local vs Global Variables

```
1  # Global variables
2  company_name = "TechCorps Analytics"
3  tax_rate = 0.08
4
5  def calculate_total_cost(subtotal):
6      """Calculate total cost including tax.
7
8      Args:
9          subtotal (float): Cost before tax
10
11     Returns:
12         float: Total cost including tax
13     """
14     # Local variable
15     tax_amount = subtotal * tax_rate # Uses global tax_rate
16     total = subtotal + tax_amount    # Local variable
```

Lambda Functions

Concise Anonymous Functions

```
1 # Traditional function
2 def calculate_commission(sales):
3     return sales * 0.05
4
5 # Lambda equivalent
6 calc_commission = lambda sales: sales * 0.05
7
8 # Using lambdas with built-in functions
9 sales_data = [12000, 15000, 8000, 22000, 18000, 9500]
10
11 print("Sales Rep Performance:")
12 print("-" * 30)
13
14 # Map - apply function to all elements
15 commissions = list(map(lambda x: x * 0.05, sales_data))
16 for i, (sales, commission) in enumerate(zip(sales_data, com
```


Business Application Example

Customer Loyalty Program System

```
1  def determine_loyalty_tier(total_spent, years_active):
2      """Determine customer loyalty tier based on spending and
3
4      Args:
5          total_spent (float): Total amount spent by customer
6          years_active (int): Years as active customer
7
8      Returns:
9          tuple: (tier, discount_rate, benefits)
10
11     if total_spent >= 10000 and years_active >= 3:
12         return "Platinum", 0.15, ["Free shipping", "24/7 su
13     elif total_spent >= 5000 and years_active >= 2:
14         return "Gold", 0.10, ["Free shipping", "Priority su
15     elif total_spent >= 1000 or years_active >= 1:
16         return "Silver", 0.05, ["Free shipping on orders >8
17     else:
18         return "Bronze", 0.00, ["Standard benefits"]
```


Combining Concepts

Summit Exports Container System

```
1  def create_container_record(container_id, route, cargo_value):
2      """Create immutable container record using tuple."""
3      origin, destination = route # Tuple unpacking
4      return (container_id, origin, destination, cargo_value)
5
6  def track_destinations(containers):
7      """Track unique destinations using set."""
8      destinations = set()
9
10     for container in containers:
11         _, _, destination, _, _ = container # Unpack tuple
12         destinations.add(destination)
13
14     return destinations
15
16  def calculate_route_statistics(containers):
17      """Calculate statistics for container operations."""
18     if not containers:
```


Key Takeaways

What You've Learned Today

1. **Tuples** provide immutable, ordered data storage for fixed records

Next Week Preview

We'll dive deeper into **advanced functions**, **modules**, and **importing libraries** to expand your Python toolkit even further!

What You've Learned Today

1. **Tuples** provide immutable, ordered data storage for fixed records
2. **Sets** excel at unique collections and mathematical operations

Next Week Preview

We'll dive deeper into **advanced functions**, **modules**, and **importing libraries** to expand your Python toolkit even further!

What You've Learned Today

1. **Tuples** provide immutable, ordered data storage for fixed records
2. **Sets** excel at unique collections and mathematical operations
3. **Functions** enable code reusability and modular programming

Next Week Preview

We'll dive deeper into **advanced functions**, **modules**, and **importing libraries** to expand your Python toolkit even further!

What You've Learned Today

1. **Tuples** provide immutable, ordered data storage for fixed records
2. **Sets** excel at unique collections and mathematical operations
3. **Functions** enable code reusability and modular programming
4. **Variable scope** controls access to data within programs

Next Week Preview

We'll dive deeper into **advanced functions**, **modules**, and **importing libraries** to expand your Python toolkit even further!

What You've Learned Today

1. **Tuples** provide immutable, ordered data storage for fixed records
2. **Sets** excel at unique collections and mathematical operations
3. **Functions** enable code reusability and modular programming
4. **Variable scope** controls access to data within programs
5. **Lambda functions** offer concise solutions for simple operations

Next Week Preview

We'll dive deeper into **advanced functions**, **modules**, and **importing libraries** to expand your Python toolkit even further!

What You've Learned Today

1. **Tuples** provide immutable, ordered data storage for fixed records
2. **Sets** excel at unique collections and mathematical operations
3. **Functions** enable code reusability and modular programming
4. **Variable scope** controls access to data within programs
5. **Lambda functions** offer concise solutions for simple operations
6. **Business applications** benefit from combining all these concepts

Next Week Preview

We'll dive deeper into **advanced functions**, **modules**, and **importing libraries** to expand your Python toolkit even further!

Practice Exercises

Try These Challenges:

1. **Product Catalog Management**

- ▶ Create tuples for product information
- ▶ Use sets to find unique categories
- ▶ Write functions to calculate totals

2. **Customer Analysis**

- ▶ Build customer records with tuples
- ▶ Track unique customer segments with sets
- ▶ Create functions for loyalty calculations

3. **Inventory Optimization**

- ▶ Use tuples for SKU information
- ▶ Sets for supplier management
- ▶ Functions for reorder point calculations

Questions?

Resources for Practice

- ▶ Course materials and assignments
- ▶ Summit Exports case study
- ▶ Cozy Coffee Corner company exercises
- ▶ Office hours for personalized help

! Important

Remember: Functions are the foundation of all advanced programming. Practice creating small, focused functions!

Contact Information

- ▶ Email: smith515@usf.edu
- ▶ Office Hours: See Canvas for current schedule